

Algorithmic Family vs. Preprocessing Pipeline: Evaluating Non-Transferability and Model Dependency in Imbalanced Datasets

Fermín Barrena, Aimar Alustiza

Repository: <https://github.com/THeFeRTXo/Research-Project-Data-Mining>

Paper Publication Date: 23rd May 2026

Abstract- This research investigates the structural dependencies between machine learning classifiers and preprocessing pipelines using the highly imbalanced Bank Marketing dataset [1]. While literature frequently treats data preprocessing as a universal, model-agnostic step, this study introduces a two-phase methodology to evaluate pipeline non-transferability across divergent architectures: the "distance family" (KNN and Logistic Regression) versus the "partition family" (Decision Trees and Random Forest). Phase 1 identifies the optimal pipeline for each classifier via extensive GridSearch optimized for F1-Score. Phase 2 implements a cross-assignment "swap protocol" to measure performance degradation under opposing environments. Empirical results reveal severe algorithmic incompatibilities: the distance family relies on PCA and undersampling (NCR) for geometric clarity, whereas the partition family rejects PCA, and Random Forest leverages oversampling (SMOTE). Under the swap protocol, KNN suffered a drastic F1-Score drop (-0.0753) due to SMOTE's localized noise. We conclude that preprocessing is an intrinsic, non-transferable extension of the classifier architecture. Finally, Random Forest emerged as the optimal model (F1-Score: 0.5898); its preservation of the original feature space enabled the integration of SHAP values to deliver regulatory-compliant global and local explainability.

I. DATASET DESCRIPTION

For this research project, the bank-full.csv dataset was used [1], detailing Portuguese direct marketing campaigns. The objective is predicting fixed-term deposit subscriptions (target variable y). The dataset comprises 45,211 records and 17 variables with no explicit nulls, though categorical "unknown" labels necessitate targeted imputation. The primary methodological challenge is severe class imbalance (Figure 1): the majority class ('no') represents 88.3% (39,922 cases), while the minority class ('yes') constitutes only 11.7% (5,289 cases).

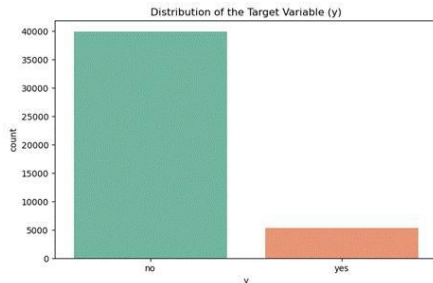


Figure 1: Distribution of the Target Variable (y) in the bank-full dataset.

The Bank Marketing dataset [2] is widely used to predict if a customer will sign up for a deposit. Most studies focus on comparing different models, treating data preprocessing as a basic, one-size-fits-all step. For example, Nandini et al. [3] tested different SMOTE techniques to improve a Logistic Regression model. While their results showed that different methods improved accuracy (reaching 94% with K-Means SMOTE), the varying results suggest that no single data setup works perfectly in all situations.

However, a major problem remains. Even if researchers know different techniques yield different results, they usually apply the exact same preprocessing pipeline to all their classifiers. Data preparation is still treated as a generic cleaning step, ignoring the fact that different algorithms use completely different math (like distance-based K-Nearest Neighbors versus partition-based Decision Trees).

This project is designed to address this specific gap. Instead of just comparing model scores using the same static data setup, we investigate how pipelines fail when swapped between different algorithm families. By showing how a perfect pipeline for one model can actually ruin another, we aim to prove that preprocessing must be treated as a core, dynamic part of the algorithm itself.

II. RESEARCH OBJECTIVES

The main objective of this study is to analyze the structural dependencies between machine learning algorithms and their preprocessing environments. We postulate that a dataset optimized for one family of algorithms will be deficient for a fundamentally different family.

To guide our experimentation and analysis, we have formulated the following two research questions:

- Research Question 1 (RQ1): How does the cross-assignment of optimal preprocessing pipelines between the "distance family" (KNN and Logistic Regression) and the "partition family" (Decision Trees and Random Forest) impact classifier performance?

- Research Question 2 (RQ2): To what extent do the preprocessing methods selected by the winning model limit or enhance its explainability?

III. METHODOLOGY

Note: All experimental implementations, pipelines, and results discussed in this paper are executed within the single repository notebook `research_code.ipynb`. References to specific findings are mapped directly to their corresponding execution cells below.

A. Experiment Structure and Metrics

We have divided the research into two different phases with our four models (two per family). In Phase 1, we identified the optimal preprocessing pipeline and the optimal hyperparameters for each classifier. In Phase 2, we execute a swap protocol. We freeze the pipelines of all classifiers and force the models of the opposite family to make predictions on them. This way, we measure the real impact of the compatibility between families.

To evaluate success, we completely discarded Accuracy. Our banking dataset is quite imbalanced. Using accuracy would generate a false sense of success. Therefore, we exclusively used the F1-Score [4]. This metric penalizes false positives and false negatives in the minority class.

B. Search Space and Combinatorics

We designed a preprocessing architecture divided into a static and a dynamic stage.

In the static stage, we applied `SimpleImputer(strategy='most_frequent')` and an `OrdinalEncoder` to all categorical variables. This design decision is deliberate. First, we wanted to avoid a gigantic combinatorics space. We also wanted to avoid smoothing out the results. We know that `OrdinalEncoder` assigns numerical values (1, 2, 3) to categories with no real order. For trees, this is a comfortable environment. For the distance family, it's an uncomfortable starting point [5]. The goal is to force each classifier to "take a stand" within the pipeline, to choose techniques to generate different pipelines.

For the dynamic part, we configured a massive `GridSearchCV` [6]. We chose enough methods so that the customization is broad, but without creating an unmanageable computational load. We configured our massive `GridSearchCV` to evaluate a comprehensive set of techniques grouped into four main preprocessing stages. For each category, we included passthrough as a baseline option. It is important to clarify that passthrough is not a transformation method itself, but rather an explicit configuration that allows the algorithm to opt out of that particular preprocessing step, thereby leaving the data unaltered at that specific stage.

- Outlier Considerations: We evaluated the impact of handling anomalous data using **IQR** and **MeanStd** (methods custom-coded by us to detect outliers based on percentiles and standard deviation, respectively), alongside the passthrough option.
- Scaling and Normalization: We tested **StandardScaler** and **MinMaxScaler** to manage the numerical scale of the features, or passthrough to leave original scales intact.

- Data Balancing: We experimented with **SMOTE** (for oversampling) and **NeighbourhoodCleaningRule** (for undersampling) to address the minority class, as well as a passthrough option to preserve the original distribution.
- Dimensionality Reduction: Placed strictly as the final preprocessing step subsequent to encoding, scaling, and balancing the pipeline offered **Principal Component Analysis** (PCA) to mitigate collinearity, or passthrough to retain the original feature space. Instead of testing an arbitrary number of components, PCA was parametrized dynamically using a fixed 95% explained variance threshold (`n_components=0.95`). This threshold is a robust statistical heuristic chosen to optimally filter noise and multicollinearity while preserving critical predictive variance [7].

Importantly, these preprocessing steps are not tested one by one. Instead, `GridSearchCV` finds the best combination of data preparation techniques and model settings at the same time. This ensures that the algorithm is perfectly tuned to get the best possible results from the data.

IV. EXPERIMENT 1: OPTIMIZATION OF ENVIRONMENTS

In Phase 1, we analyzed the best preprocessing pipelines found by `GridSearchCV`, which are summarized in Figure 2 (generated in Cell {13}). This figure shows the optimal steps (Outliers, Scaling, Sampling, and PCA) and the highest F1-Score for each model. The results show a clear pattern: the classifiers did not choose these steps randomly. Instead, their choices directly match the mathematical logic of each algorithm.

Model	Outliers	Scaling	Sampling	PCA	F-score
KNN	Not applied	StandardScaler	NCL	PCA (95%)	0.507
Logistic Regression	Not applied	StandardScaler	NCL	PCA (95%)	0.5249
Decision Tree	IQR	MinMaxScaler	NCL	Not applied	0.561
Random Forest	Not applied	MinMaxScaler	SMOTE	Not applied	0.5898

Figure 2: Optimal preprocessing pipelines selected by each classification model.

A. KNN and Logistic Regression Results

In our study, KNN and Logistic Regression make up the 'distance family'. When analyzing the Phase 1 results, we found a very interesting pattern: both models independently chose the exact same preprocessing pipeline. They both decided to keep outliers (passthrough), use `StandardScaler`, balance the data with `NeighbourhoodCleaningRule`, and apply PCA for dimensionality reduction (KNN results are in Cell {9}, and Logistic Regression results are in Cell {10}).

To understand why both models chose the exact same pipeline, we must look at how they work internally. We hypothesize that this specific combination of steps perfectly fixes the mathematical weaknesses of distance-based algorithms:

- Why did both models choose StandardScaler? For KNN, it is a strict geometric requirement; without uniform scales, a variable measured in thousands would disproportionately weight the Euclidean distance calculations, eclipsing smaller metrics [8]. For Logistic Regression, uniform scaling is necessary for its built-in penalty system (L2 regularization). It ensures that the model treats all variables equally, regardless of their original size.
- Why was Neighbourhood Cleaning Rule (NCL) chosen? Distance-based models need very clear boundaries between classes to make good predictions. NCL helps by removing confusing or overlapping data points from the majority class. This creates a clean separation between classes without adding fake data points, which could ruin the model's distance calculations.
- Why was PCA chosen? In financial datasets, numerical variables like age, balance, and call duration are often highly correlated. PCA acts as a mathematical filter for this overlapping noise. It transforms the data to provide the models with a clean and independent set of features, making it much easier for them to learn.
- Why did the models choose to keep outliers (passthrough)? In banking, extreme values are rarely mistakes. Instead, they often represent real, high-value customer behaviors. Keeping these extreme data points gives the models valuable clues for making predictions without breaking their mathematical structure.

B. Decision Tree and Random Forest Results

Decision Trees (DT) and Random Forests (RF) make up the 'partition family'. Unlike the distance models, DT and RF did not choose the exact same pipeline. They both agreed on rejecting PCA (passthrough) and using MinMaxScaler for scaling. However, they chose completely different methods for handling outliers and balancing the data (see Cell {11} for the DT setup and Cell {12} for RF).

These selected preprocessing parameters can be justified by the architectural differences between a single tree and an ensemble method:

- "Why did these models reject PCA? Tree-based models work by making decisions based on single, clear variables (like age or balance). PCA mixes all these original variables together into abstract components. This blending makes it much harder for the tree to find clean, simple rules to separate the data.
- Why did these models choose MinMaxScaler? Tree-based models make decisions using simple thresholds (like asking if a value is greater or less than a certain number). Because of this, changing the scale of

the variables doesn't really affect how they work. Choosing MinMaxScaler instead of skipping scaling (passthrough) wasn't a strict requirement, but rather a neutral mathematical choice that didn't change the model's structure."

- Why did the Decision Tree remove outliers using IQR while the Random Forest kept them? A single Decision Tree is very sensitive to extreme values. If outliers are not removed, the tree might waste entire branches trying to separate these unusual data points [9]. On the other hand, a Random Forest is much stronger because it combines many trees. By averaging their predictions, the Random Forest can naturally handle outliers without needing to delete any data.
- Why did Random Forest choose SMOTE while the Decision Tree chose NCL? A single Decision Tree uses undersampling (NCL) to create simple, general rules. However, a Random Forest needs as much data diversity as possible to build different, independent trees. Removing data through undersampling would hurt this process. Instead, SMOTE adds synthetic examples, giving the Random Forest the diverse data it needs to grow strong [10]. This perfectly explains why Random Forest achieved the highest F1-Score of our experiment (0.5898).

V. EXPERIMENT 2: VALIDATION OF NON-TRANSFERABILITY (SWAP)

In Phase 2, we tested model compatibility by forcing them to use the opposing family's optimal pipeline. To ensure a fair test, we froze all the preprocessing steps but allowed each model to re-optimize its own hyperparameters. This guarantees that any drop in performance is caused entirely by using the wrong pipeline, not by poor model settings.

A. Empirical Results

Combinatorial Hypothesis Validation: Total Cross (4x2)				
	1. Natural Matrix (4x1) Own Pipeline	2. Swap Matrix (4x2) Models in Enemy Habitats		3. Difference Matrix (4x2) Non-Transferability Impact
KNN	0.5070	0.4981 (vs Tree)	0.4917 (vs RF)	-0.0187 (vs Tree) -0.0751 (vs RF)
Logistic Regression	0.5249	0.5123 (vs Tree)	0.5181 (vs RF)	-0.0127 (vs Tree) -0.0069 (vs RF)
Decision Tree	0.5610	0.4989 (vs KNN)	0.4989 (vs LR)	-0.0621 (vs KNN) -0.0621 (vs LR)
Random Forest	0.5898	0.5400 (vs KNN)	0.5400 (vs LR)	-0.0497 (vs KNN) -0.0497 (vs LR)
	Own Habitat	Enemy Pipeline A	Enemy Pipeline B	Drop (Pipeline A) Drop (Pipeline B)

Figure 3: F1-Score results matrix evaluating combinatorial cross-pipeline performance.

To clearly show the impact of the swap protocol, Figure 3 (generated in Cell {19}) organizes the results into three different comparison matrices:

- Baseline Matrix (Left): This initial column shows the maximum F1 score achieved by each of the four models when operating within their optimal pipelines.
- Cross-Assignment Matrix (Center): This matrix details the raw F1-Scores obtained when each model (rows) is

evaluated using the pipelines of the opposing algorithmic family (columns). Below each F1-score, a reference indicates which opposing classifier provided the pipeline for that specific evaluation.

- Degradation Matrix (Right): This final matrix calculates the exact difference between the baseline (left) and the cross-assigned results (center), providing a direct measure of performance variance.

B. Hypotheses

The performance drops seen during the swap test show that these algorithms are fundamentally incompatible with the wrong data setups. We propose the following reasons to explain why the models failed:

- The impact of PCA. Both the Decision Tree and Random Forest lost performance (-0.0621 and -0.0497) when forced to use the distance family's pipeline (see Cell [17] for DT and Cell [18] for RF). We hypothesize that PCA is the main reason for this drop. Tree models need clear, single variables to make good decisions. Because PCA mixes all the original variables together, it hides the clear boundaries that these models rely on to split the data.
- Distance Family Degradation: KNN suffered the worst performance drop (-0.0753) when forced to use the Random Forest pipeline (see Cell [15]). We theorize this happened because of two compounding issues: the loss of PCA and the forced use of SMOTE. While SMOTE helps Random Forest by adding synthetic data points (giving it the volume it needs), it ruins KNN's logic. SMOTE creates these new points by drawing lines between existing ones. Because KNN relies entirely on measuring exact distances between real data points, these synthetic examples act as localized noise. They flood the space, destroying the accuracy of KNN's real distance calculations.
- Global vs. Local Robustness: Notably, Logistic Regression demonstrated significant resilience to the Random Forest pipeline, experiencing only a marginal drop of -0.0069 (see Cell [16]). We hypothesize that because Logistic Regression optimizes a global hyperplane across the entire dataset, it is substantially more robust to the localized geometric perturbations introduced by SMOTE, unlike the strictly localized KNN algorithm.

VI. SHAP VALUES

To understand how our models make decisions, we used SHAP (SHapley Additive exPlanations) [1]. This tool opens the algorithm's 'black box' by calculating the exact impact of each feature, like age or balance. A positive SHAP value pushes the prediction towards a 'yes' (subscribing), while a negative value pushes it towards a 'no'. This method is highly valuable because it reveals the model's logic for both overall global trends and individual customer predictions.

While Logistic Regression and KNN performed well, they completely relied on PCA to work properly. In banking, this is a major problem. Regulations like GDPR demand a 'Right to Explanation' for automated decisions. Because PCA mixes real variables (like age or balance) into abstract components, it makes the model's logic impossible to explain to a client. On the other hand, our winning model, Random Forest (F-score 0.5898), rejected PCA and kept the original variables intact. We selected Random Forest as our final choice because it not only has the best performance, but it also allows us to use SHAP to provide clear, transparent explanations to the bank.

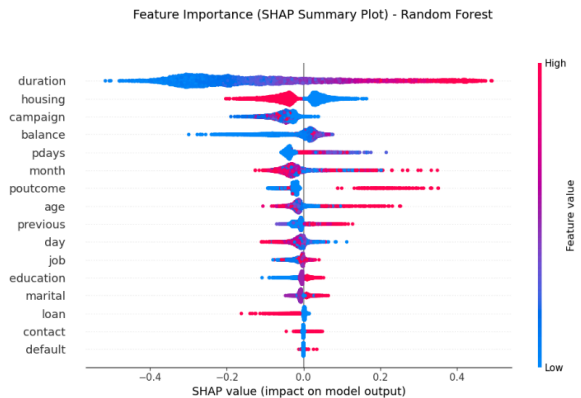


Figure 4: SHAP Summary Plot detailing global feature importance and directional impact.

Figure 4 (generated in Cell [20]) displays a SHAP summary plot revealing the most important variables for our trained model:

- Duration: This is the most critical factor. Longer calls (red dots on the right) strongly increase the chances of a customer subscribing, while very short calls (blue dots on the left) almost always lead to a rejection.
- Housing: The model shows an interesting pattern. Customers with an existing mortgage (blue dots on the right) are more likely to subscribe to the deposit, whereas those without one (red dots on the left) show a lower probability of accepting.
- Poutcome: Because our OrdinalEncoder assigned the highest numerical value to a 'success' (shown as red dots), we can clearly see that customers who accepted a previous bank offer are much more likely to say 'yes' again.

Beyond analyzing the variables on a global scale, it is highly valuable to see how the model makes decisions for individual customers. This leads to an important question: what specific characteristics of a single customer push the model to predict a 'yes' or a 'no'?

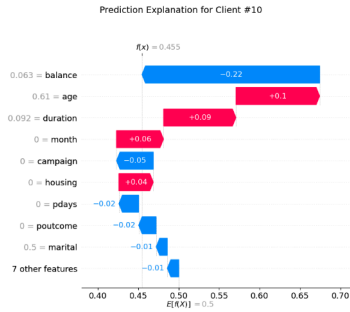


Figure 5: SHAP Waterfall Plot explaining the local prediction components for Customer #10.

Figure 5 shows a SHAP waterfall plot explaining the prediction for a single client, Customer #10 (generated in Cell {20}). The baseline starting probability is 50% (0.5). This happens because we used SMOTE to balance the data earlier, artificially raising the original 13% baseline. For this specific customer, the final chance of subscribing is 45.5%. Why? The biggest negative factor is their account balance, which drops the probability by 22%. On the positive side, their age and the call duration push the probability up by 10% and 9%, respectively. This perfectly matches our global findings, showing once again that call duration is a key positive driver.

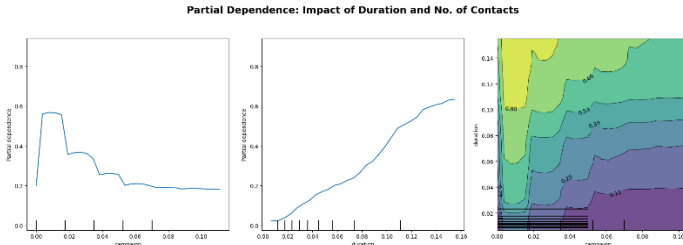


Figure 6: Partial Dependence Plots illustrating the marginal effect and interaction of 'duration' and 'campaign'.

Figure 6 (plotted in Cell {21}) displays Partial Dependence Plots (PDP), which illustrate how individual features impact the final prediction. The left graph shows that the 'campaign' variable has a negative trend: fewer contacts lead to higher success, while calling a customer too many times drastically lowers the probability. The center graph confirms that call 'duration' has a strong positive effect, where longer calls mean higher conversion rates. Finally, the contour plot on the right combines both features. It clearly highlights (in lighter green and yellow) that the optimal strategy for securing a deposit is having long conversations but keeping the total number of contacts low.

VII. CONCLUSIONS

The goal of this research was to move past basic predictive modeling and investigate how machine learning algorithms depend on their specific preprocessing pipelines. Based on the results of our swap experiments and our interpretability analysis, we can now definitively answer our main research questions:

Addressing RQ1 (Impact of Cross-Assigned Preprocessing Pipelines): Our results clearly show that there is no "universal

pipeline" in machine learning. The performance drops we saw when swapping setups were not random mistakes, but deep incompatibilities: what feeds one algorithm effectively poisons another. For example, PCA cleans up data for distance models, but it makes tree models "blind" by mixing up the clear features they need. Similarly, SMOTE provides the vital synthetic data Random Forests need to grow, but it floods KNN with fake points, ruining its real distance calculations. Ultimately, data preprocessing isn't just a cleaning step before the real work begins, it is a fundamental part of the algorithm itself.

Addressing RQ2 (Explainability Limits and SHAP Value Integration):

Our study found a major trade-off between getting the best data setup and keeping the model understandable. While Logistic Regression and KNN performed well, they completely relied on PCA to work properly. Since PCA mixes real variables into abstract mathematical components, it destroys transparency. In banking, this is a huge problem because laws like GDPR demand a 'Right to Explanation' for automated decisions. On the other hand, our winning model, Random Forest, rejected PCA and kept the original variables intact. Because of this, we were able to use SHAP to open its 'black box', providing clear, legally compliant explanations for both overall trends and individual customer predictions.

REFERENCES

- [1] "UCI Machine Learning Repository. Bank Marketing," 30 04 2026. [Online]. Available: <https://archive.ics.uci.edu/dataset/222/bank+marketing>.
- [2] S. Moro, P. Cortez, and P. Rita, "A data-driven approach to predict the success of bank telemarketing," *Decision Support Systems*, vol. 62, pp. 22–31, 2014. [Online]. Available: <https://acortar.link/UHrJxa>. [Accessed 30 04 2026].
- [3] K. R. Nandini, P. S. R. Chandra, P. S. R. Chowdary, and P. S. R. Krishna, "A Comparative Study of Re-Sampling Techniques on Machine Learning Model Performance," *Academia.edu*, Dec. 2022. Accessed: May 23, 2026. [Online]. Available: <https://acortar.link/5sHeyH>. [Accessed 30 04 2026].
- [4] G. F. a. N. V. C. Troy Raederl, "Learning from Imbalanced Data: Evaluation," 2012. [Online]. Available: <https://acortar.link/eSrlD5>. [Accessed 30 04 2026].
- [5] A. I. A. K. A. Udila, "Encoding methods for categorical data: A comparative analysis for linear models, decision trees, and support vector machines," 2023. [Online]. Available: <https://acortar.link/kLSz9N>. [Accessed 30 04 2026].
- [6] A. K. V. S. R. Ranjan G S K, "K-Nearest Neighbors and Grid Search CV Based Real Time Fault Monitoring System for Industries," 2019. [Online]. Available: <https://acortar.link/xLjyRO>. [Accessed 30 04 2026].
- [7] I. T. Jolliffe, *Principal Component Analysis*, New York: Springer, 2002.
- [8] M. Z. C. Muasir Pagan, "Investigating the impact of data scaling on the k-nearest neighbor algorithm," 2023. [Online]. Available: <https://www.iaesprime.com/index.php/csit/article/view/279/102>. [Accessed 30 04 2026].
- [9] G. H. John, "Robust Decision Trees: Removing Outliers from Databases," 1995. [Online]. Available: <https://cdn.aaii.org/KDD/1995/KDD95-044.pdf>. [Accessed 30 04 2026].
- [10] R. C. Bhagat and S. S. Patil, "Enhanced SMOTE algorithm for classification of imbalanced big-data using Random Forest," 2015. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/7154739>. [Accessed 30 04 2026].
- [11] S.-I. L. Scott M. Lundberg, "A Unified Approach to Interpreting Model," 2017. [Online]. Available: <https://acortar.link/hPXgBF>. [Accessed 30 04 2026].